

HemoTrack adaptado

Rota Vital • Projeto Integrador • Unidade 1

Java / Spring Boot e C / C++ com as mesmas operações

Documento de comparação, modelagem e tradução comentada • 17 de setembro de 2026

Esta entrega parte de duas fontes: os requisitos da Unidade 1 enviados na conversa e o arquivo HemoTrack(1).zip fornecido pelo aluno. O HemoTrack recebido apresenta a Entrega 02, com Spring Boot 3.5.0, Thymeleaf, JPA/H2, alocação automática por compatibilidade ABO/Rh e prioridade por validade. Esses últimos algoritmos não podem ser mantidos na Unidade 1.

A adaptação mantém os cadastros úteis e a referência visual, mas substitui o armazenamento de domínio por estruturas próprias. A versão Java usa nós ligados por referências e é consumida pelo Spring Boot. A versão C usa structs, ponteiros e malloc/free; uma interface C++ oferece menu de terminal com iostream/stdio e servidor HTTP para abrir as mesmas telas.

Equivalência exigida: todas as operações de negócio expostas pela aplicação Java adaptada também são expostas pela aplicação C/C++ adaptada. Ambas usam os mesmos arquivos HTML/CSS/JavaScript. Não se trata de Java chamando um processo C: cada servidor executa sua própria implementação e possui seu próprio estado.

Limite da identidade com o original: o projeto não copia literalmente todas as regras da Entrega 02, porque isso entraria em conflito com a proibição de FEFO e compatibilidade. A interface conserva a identidade azul-escura, rosa e ciano e os fluxos de cadastro, com navegação ampliada para hospitais, histórico e visualização das estruturas. As diferenças são registradas na matriz seguinte.

Identificação acadêmica: completar integrantes, turma, professor e instituição conforme o padrão da disciplina. O código originalmente desenvolvido em sala não foi fornecido; a continuidade exata com esse código precisa ser conferida pelo aluno.

Organização para avaliação

c/rota_vital.h declara os dados e contratos; c/rota_vital.c contém os algoritmos avaliados. cpp/programa.cpp adapta entrada/saída e HTTP para esse núcleo. As classes Java em src/main/java/br/edu/rotavital reimplementam os algoritmos; RotaVitalServico coordena as operações, e ApiRotaVital publica as rotas HTTP.

1. Comparação com o HemoTrack recebido

Item observado no original	Adaptação para a Unidade 1	Java e C/C++
BolsaSangue: código, hemocomponente, tipo e datas	Campos preservados; acrescentado volume e ID explícito	Mesmos dados e validações
ServicoBolsa.cadastrarLote	Lote por ID inicial; código-base com sufixo -001 etc.	Lote inteiro ou nenhuma inserção
Requisicao: hospital, tipo, componente, quantidade e urgência	Campos preservados; urgência apenas informativa	Fila FIFO em ambas
Identificadores JPA e UUID de rastreamento	ID inteiro informado; pedido exibido como #ID	Mesma regra, sem UUID
Repositórios JPA e H2 em arquivo	Nós próprios em memória	Estado perdido ao encerrar
listarTodas ordena bolsas por validade	Ordem da lista encadeada, com inserção no início	Nenhuma ordenação por validade
ServicoCompatibilidade.compativel	Serviço e tabela de compatibilidade excluídos	Nenhuma regra ABO/Rh
ServicoAlocacao.alocar: seleção automática FEFO	Seleção manual de IDs pelo operador	Conclusão integral da primeira requisição
Alocação parcial e estado reservado	Fluxo simplificado: pendente ou concluído	Consumo integral, registrado na pilha
Remover pedido em qualquer posição	Cancelar somente a primeira requisição	Preserva FIFO
Remoção recalcula alocações e devolve reservas	Não há reservas automáticas nesta unidade	Retirar histórico não desfaz conclusão
Telas Thymeleaf específicas do Java	Telas estáticas compartilhadas por ambos os servidores	Mesmos arquivos de interface
Cadastro de hospitais inicializado por configuração	Cadastro interativo em ABB	Inserir, consultar, remover e percorrer

Evidência no código original: `ServicoAlocacao.buscarBolsasCompativeisDisponiveis` aplica um filtro por `ServicoCompatibilidade` e ordena por `BolsaSangue.getDataValidade`. `ServicoBolsa.listarTodas` também ordena por validade. `ServicoRequisicao` usa UUID e repositórios; não implementa uma fila ligada por nós.

O original utiliza `EnumMap` na compatibilidade. Isso não deve ser confundido com uma implementação própria de tabela hash; a exclusão desse serviço decorre da proibição da compatibilidade ABO/Rh. O domínio adaptado não usa `HashMap`, `unordered_map` ou outra tabela hash para guardar bolsas, pedidos ou hospitais.

2. Entidades e contratos comuns

Entidade	Campos principais	Estrutura
Bolsa	ID, código, tipo, volume, hemocomponente, coleta e validade	Lista simplesmente encadeada
Requisicao	ID, hospitalId, tipo, quantidade, hemocomponente e urgência	Fila com início e fim
Operacao	ID da requisição, hospitalId, quantidade e tipo solicitado	Pilha de conclusões
Hospital	ID, nome e cidade	Árvore binária de busca

Os IDs são inteiros positivos. O código da bolsa é obrigatório e único entre bolsas atualmente no estoque; sua comparação é exata. Nome e cidade também são obrigatórios. Java verifica os limites em bytes UTF-8 para corresponder aos vetores de char do C: código até 47 bytes, nome até 79 e cidade até 59, além do terminador nulo em C.

As datas devem existir no calendário, usar AAAA-MM-DD e ter validade posterior à coleta. Guardar datas e conferir sua consistência não implementa FEFO: elas não são usadas para ordenar, escolher ou recomendar bolsas. Tipo, hemocomponente e urgência são metadados; a conclusão não calcula compatibilidade nem prioridade.

A quantidade do pedido é medida em bolsas, enquanto volumeMI/volume_ml descreve o volume individual. O lote aceita de 1 a 1000 bolsas e gera IDs consecutivos a partir do ID inicial. Com várias bolsas, acrescenta -001, -002 e assim por diante ao código-base. Colisão de qualquer ID ou código recusa o lote inteiro.

Hospital referenciado por pedido pendente ou registro de histórico não pode ser removido. O ID de uma requisição não pode repetir outro pedido pendente nem um registro ainda presente no histórico. Após remover esses registros, não há um catálogo permanente de IDs já usados. A aplicação é acadêmica e em memória.

Retornos: C usa Resultado e um parâmetro de saída. Java usa boolean/Optional nas estruturas e FalhaDominio na camada de serviço. Os adaptadores HTTP convertem ausência em 404, conflito em 409 e entrada inválida em 400. Os textos das mensagens podem diferir; os resultados de negócio e o estado são equivalentes.

3. Estoque: equivalência da lista

C: NoBolsa, ListaEstoque e lista_inserir/lista_consultar/lista_remove em c/rota_vital.c. **Java:** NoBolsa e inserir/consultar/remover em ListaEstoque.java.

O nó guarda a bolsa e a ligação para o próximo. C possui NoBolsa *inicio; Java possui NoBolsa inicio. Na inserção, ambas as versões procuram ID e código duplicados antes de ligar o novo elemento. O novo nó é colocado no início da cadeia.

```
novo->dado = bolsa;  
novo->proximo = lista->inicio;  
lista->inicio = novo;  
  
inicio = new NoBolsa(bolsa, inicio);
```

Justificativa amarrada ao código: o construtor Java guarda dado e proximo; por isso, a expressão acima reproduz as três atribuições de C. O antigo início passa a ser o segundo nó, e o restante da cadeia permanece alcançável. Inserir 101, 102 e 103 produz 103, 102, 101 em ambas as versões. Nenhuma usa uma coleção pronta para armazenar o estoque.

Na remoção em C, NoBolsa **ligacao aponta inicialmente para o campo inicio e depois para o proximo de um nó anterior. A atribuição *ligacao = removido->proximo retira o nó da cadeia. Java usa anterior e atual: altera inicio quando remove o primeiro; nos demais casos, faz anterior.proximo = atual.proximo. O efeito sobre a cadeia é o mesmo. C executa free; Java deixa o nó desligado disponível para coleta de lixo.

Lote: lista_inserir_lote e inserirLote preparam uma cadeia temporária. Cada item é validado contra a lista original e contra a cadeia temporária. Se houver erro, o C libera a cadeia; Java abandona as referências temporárias. Só depois do sucesso o último nó temporário aponta para o início antigo e o início principal é atualizado. Isso evita inserir parcialmente um lote.

Consultar/remover por ID custa $O(n)$. A ligação de um nó custa $O(1)$, mas a inserção pública é $O(n)$ por causa da verificação de duplicidade. Um lote de k itens custa $O(kn + k^2)$. O método todos() de Java produz apenas uma cópia em vetor para JSON; o vetor não substitui o armazenamento por nós.

4. Requisições: equivalência da fila

C: FilaRequisicoes, fila_inserir/fila_consultar/fila_remove. **Java:** FilaRequisicoes.inserir/consultar/remove e descartarInicio.

A fila possui início e fim. O novo nó sempre tem proximo vazio. Se há um fim anterior, ele passa a apontar para o novo nó; se a fila está vazia, inicio também recebe o novo. A atualização de fim ocorre nos dois casos.

```
if (fila->fim) fila->fim->proximo = novo;
else fila->inicio = novo;
fila->fim = novo;
```

```
if (fim == null) inicio = novo;
else fim.proximo = novo;
fim = novo;
```

Justificativa amarrada ao código: as condições estão escritas em ordens diferentes, mas tratam as mesmas duas situações. Depois da operação, o novo nó é o último e todos os pedidos anteriores mantêm sua ordem relativa. Na remoção, ambas avançam inicio para inicio.proximo. Quando sai o último nó, ambas zeram fim para permitir a próxima inserção sem referências inválidas.

O menu e as APIs só concluem ou cancelam a primeira requisição. O usuário informa o ID que espera processar; a camada de serviço confere se ele ainda está na frente. Essa verificação impede que uma tela desatualizada processe acidentalmente outro pedido. O campo urgência é mantido para correspondência com o HemoTrack, mas não altera a ordem FIFO.

A validação de existência do hospital acontece em rota_solicitar/RotaVitalServico.solicitar, antes da chamada à fila. A fila isolada cuida do encadeamento e das duplicidades; ela não depende da ABB ou de Spring. Essa separação permite testar as estruturas independentemente da aplicação.

Consultar e retirar a frente custam $O(1)$. Acrescentar a ligação no fim custa $O(1)$; a operação pública completa custa $O(n)$, devido à busca de ID repetido. A limpeza em C retira e libera todos os nós. Java remove as referências de início e fim em limpar().

5. Histórico: equivalência da pilha

C: PilhaHistorico, pilha_inserir/pilha_consultar/pilha_remove. **Java:** PilhaHistorico.inserir/consultar/remove.

A pilha registra conclusões de requisições. Cada nó contém uma Operacao e o acesso ao nó anterior na sequência. O topo aponta sempre para a conclusão mais recente. Após concluir os pedidos 1 e 2, a consulta devolve 2 e as retiradas são 2, 1.

```
novo->dado = operacao;  
novo->proximo = pilha->topo;  
pilha->topo = novo;
```

```
topo = new NoOperacao(Objects.requireNonNull(operacao), topo);
```

Justificativa amarrada ao código: o construtor de NoOperacao em Java recebe o registro e o topo antigo. A atribuição final faz o novo nó ocupar a mesma posição que ocuparia após as três linhas em C. Os registros anteriores continuam ligados pelo campo proximo. Na retirada, ambos guardam o dado atual e avançam topo para proximo. C libera o nó retirado; Java não exige liberação explícita.

A API de remoção do topo recebe o ID da requisição esperado e confere se ele ainda corresponde ao topo. Não é permitido escolher um registro no meio da pilha. A aplicação informa que remover um registro não repõe bolsas nem desfaz uma conclusão. Essa é uma operação da estrutura, não uma implementação de desfazer/refazer.

O registro resume a requisição concluída, não armazena uma trilha clínica completa de cada bolsa utilizada. Essa limitação existe nas duas versões. Cadastros e consultas não são empilhados automaticamente: histórico significa, neste domínio, histórico de conclusões.

Inserir, consultar e retirar o topo custam $O(1)$. Produzir a lista visual do histórico exige percorrer todos os nós em $O(n)$. A pilha de histórico é diferente da pilha de chamadas usada pela recursão: a primeira é implementada explicitamente; a segunda é mantida pelo ambiente de execução.

6. Hospitais: ABB e recursão

C: `inserir_no`, `consultar_no`, `remover_no`, `percorrer_no` e `contar_no`. **Java:** `inserirNo`, `consultarNo`, `removerNo`, `percorrerNo` e `contarNo` em `ArvoreHospitais.java`.

O hospital é ordenado pelo ID: menor à esquerda, maior à direita. Cada nó tem no máximo dois filhos; portanto, a ABB também é uma árvore binária. Não há necessidade de outra árvore sem função no domínio para demonstrar o conceito de árvore binária.

Equivalência da inserção e da busca: `inserir_no` recebe o endereço da ligação que será alterada em C. Java retorna a nova raiz da subárvore para atribuí-la a esquerda, direita ou raiz. Assim, `inserir_no(&no->esquerda, hospital)` corresponde a `no.esquerda = inserirNo(no.esquerda, hospital)`. O caso base é uma ligação vazia, na qual se cria um nó. A busca para no nó encontrado ou quando chega a uma ligação vazia.

Equivalência da remoção: ambos tratam folha, nó com um filho e nó com dois filhos. Uma folha deixa uma ligação vazia; um filho ocupa a posição do pai removido. Com dois filhos, o código procura o menor ID da subárvore direita, copia o hospital completo para o nó atual e remove a ocorrência antiga do sucessor. Copiar somente o ID deixaria nome/cidade incorretos; as duas versões copiam o dado inteiro.

```
no->dado = sucessor->dado;  
remover_no(&no->direita, sucessor->dado.id, &descartado);
```

```
no.dado = sucessor.dado;  
no.direita = removerNo(no.direita, sucessor.dado.id());
```

Para IDs 50, 30, 70, 20, 40, 60, 80, a pré-ordem é 50, 30, 20, 40, 70, 60, 80; em-ordem é 20, 30, 40, 50, 60, 70, 80; pós-ordem é 20, 40, 30, 60, 80, 70, 50. Os testes conferem esses resultados e removem 20, 30 e 50 para verificar os três casos.

A contagem recursiva usa zero para árvore vazia e $1 + \text{esquerda} + \text{direita}$ no caso não vazio. O C libera a árvore em pós-ordem para não acessar filhos de um nó já liberado. A árvore não é balanceada: operações custam $O(h)$, podendo atingir $O(n)$ no pior caso. Percursos custam $O(n)$, com $O(h)$ de pilha de chamadas.

7. Fluxo manual e consistência

A conclusão de um pedido combina lista, fila e pilha. A entrada é o ID da primeira requisição e um vetor de IDs de bolsas escolhidas manualmente. Esse vetor transporta a escolha do operador; ele não é uma estrutura de estoque alternativa.

1. Consultar o início da fila e conferir o ID esperado.
2. Conferir se o número de IDs corresponde à quantidade solicitada.
3. Procurar cada bolsa na lista e rejeitar IDs repetidos no próprio pedido.
4. Preparar o registro na pilha de histórico.
5. Remover da lista as bolsas indicadas e retirar a primeira requisição.
6. Devolver o registro de conclusão.

C: `rota_atender` executa todas as verificações antes de alterar as estruturas. `pilha_inserir` é a última alocação de nó necessária. Se `malloc` falha nessa etapa, retorna `SEM_MEMORIA` e preserva lista e fila. Depois disso, as remoções apenas religam nós e executam `free`.

Java: `RotaVitalServico.concluirPrimeira` copia o vetor de IDs, valida quantidade, repetição e existência e cria `Operacao` antes da retirada. Em seguida, chama `historico.inserir`, `estoque.remove` para cada ID e `requisicoes descartarInicio`. Os métodos da fachada são `synchronized`, impedindo duas conclusões simultâneas sobre a mesma instância.

A detecção de repetição usa comparação direta entre os IDs do pedido, com dois laços; não utiliza hash. Não há filtro por tipo sanguíneo ou hemocomponente nem ordenação por validade. Os testes incluem tipos diferentes para comprovar que a regra proibida não foi introduzida. O sistema não confirma adequação clínica de uma escolha.

O adaptador HTTP C++ protege a chamada ao núcleo com mutex. As estruturas C isoladas pressupõem execução sequencial ou proteção externa. Java e C++ garantem exclusão mútua somente dentro da própria instância/processo; não implementam coordenação distribuída.

Erros de domínio deixam o estado inalterado nos cenários testados. Falhas fatais de processo/JVM ou erros de rede após uma mutação não equivalem a uma transação persistente; não há banco, recuperação de desastre ou promessa de entrega exatamente uma vez. O modelo é uma simulação acadêmica em memória.

8. Interfaces e mapa de operações equivalentes

Operação	Java	Núcleo C / interface C++
Inserir bolsa	ListaEstoque.inserir	lista_inserir
Inserir lote	RotaVitalServico.cadastrarLote / inserirLote	lista_inserir_lote
Consultar / remover bolsa	consultar / remover	lista_consultar / lista_remover
Enfileirar pedido	solicitar / FilaRequisicoes.inserir	rota_solicitar / fila_inserir
Consultar / cancelar primeiro	consultarPrimeira / cancelarPrimeira	fila_consultar / fila_remover com ID conferido
Concluir primeiro	concluirPrimeira	rota_atender
Consultar / retirar histórico	consultarTopo / removerTopo	pilha_consultar / pilha_remover
Inserir / consultar hospital	cadastrarHospital / consultarHospital	arvore_inserir / arvore_consultar
Remover hospital	removerHospital	rota_remover_hospital / arvore_remover
Percorrer hospitais	percorrer	arvore_percorrer
Visualizar estado	estado	Aplicacao.responder percorre os nós e serializa

ApiRotaVital expõe as operações pelo Spring Boot. Aplicacao::responder em cpp/programa.cpp expõe as mesmas rotas e campos JSON. As duas implementações fornecem /api/estado para as telas, além de rotas de bolsas, lotes, hospitais, requisições e histórico. O script `testar_equivalencia_http.py` usa ambas como aplicações reais.

A interface C++ inclui `iostream` para entrada/saída e `stdio.h` para `printf`. Seu modo `--terminal` chama o mesmo adaptador de operações que o modo HTTP. O núcleo é compilado como C11 com `gcc`; a interface é compilada como C++17 com `g++`. Não se deve compilar o arquivo C como C++ apenas por trocar sua extensão: os compiladores têm regras distintas.

Separação das bibliotecas: `cpp-httpplib` e `nlohmann/json` são usados apenas para servidor HTTP, leitura/escrita de JSON e saída estruturada do menu. As bibliotecas estão incluídas com licenças. Seus detalhes internos e os do Spring não substituem as estruturas ensinadas: bolsas, pedidos, histórico e hospitais permanecem nos nós explícitos escritos para o projeto.

9. Testes e evidências

Java: 11 testes JUnit aprovados, sem falhas e sem testes ignorados. Cobrem lista, FIFO, LIFO, três percursos, três casos de remoção na ABB, validação de entidades, lote atômico, referências de hospital, ausência de compatibilidade e duas tentativas concorrentes de concluir a mesma requisição.

C: 119 verificações funcionais aprovadas e 155 na execução instrumentada, que acrescenta contagem de liberações e falha de alocação. A contagem de nós pendentes ao final é zero. AddressSanitizer e UndefinedBehaviorSanitizer não emitiram diagnósticos nos cenários executados. LeakSanitizer está desabilitado por limitação do ambiente; a contagem de nós é uma verificação distinta.

Paridade HTTP: 56 operações executadas sobre os dois servidores. Para cada operação, o teste confere o código esperado, compara os dados de sucesso e consulta /api/estado para comparar todas as estruturas. Mensagens textuais de erro não precisam ser idênticas; comportamento e estado precisam.

A sequência inclui fila/pilha vazias, inserções em ABB, percursos, remoção de folha/um filho/dois filhos, duplicidades, lote parcialmente conflitante, datas inválidas, pedido de hospital inexistente, proteção de referências, IDs repetidos na seleção, bolsa ausente, conclusão, cancelamento e retirada LIFO. Ao final, ambas as aplicações ficam vazias.

Terminal C++: cenário executado com cadastro de hospital, lote de duas bolsas, inserção de requisição, consulta da fila, conclusão manual, consulta e retirada do histórico, remoção de hospital e estado final vazio. O registro completo está em teste_terminal_cpp.txt.

As evidências estão em documentacao: testes_c.txt, arquivos de resultado JUnit, resultado_equivalencia.txt e teste_terminal_cpp.txt. O script de paridade modifica dados de teste; deve ser executado com as aplicações inicialmente vazias. O README informa os comandos de reprodução.

Telas: os fluxos de exemplo, conclusão FIFO, seleção manual, histórico, cadastro em lote e percursos foram executados em navegador automatizado contra os dois servidores. Não ocorreram erros JavaScript, e a tela móvel foi verificada sem transbordamento horizontal. As capturas tela_java.png, tela_cpp.png e estruturas correspondentes mostram as interfaces reais.

Plataforma: compilação e execução verificadas em Linux com Java 17 e compiladores C/C++. Os scripts Windows são fornecidos para uso com Java e MinGW-w64, mas não foram executados numa máquina Windows nesta sessão.

10. Execução e checklist de entrega

Java pronto: execute EXECUTAR_JAVA.bat ou `java -jar executar/hemotrack-u1.jar` e abra `http://127.0.0.1:8080`. O JAR inclui as dependências; manter o terminal aberto mantém o servidor em execução.

C/C++: execute `compilar_cpp.bat` no Windows ou `bash compilar_cpp.sh` no Linux/WSL. Depois, `build-cpp/hemotrack_cpp` abre as telas em `http://127.0.0.1:8081`; com `--terminal`, abre o menu de avaliação. Na sintaxe Windows, utilize barras invertidas e a extensão `.exe`.

Os dois servidores iniciam com estado vazio e não compartilham dados. Para comparar, use o mesmo cenário em ambos. O botão Carregar exemplo chama as mesmas APIs de cadastro em cada servidor; não contorna o núcleo das estruturas. A interface fica disponível somente no endereço local por padrão.

Requisito	Evidência
Fonte em C com malloc/free e ponteiros	<code>c/rota_vital.h</code> e <code>c/rota_vital.c</code>
Execução C++ com iostream e stdio	<code>cpp/programa.cpp</code> , terminal e servidor independentes
Reimplementação Java integrada ao Spring Boot	Classes de estruturas, serviço, API, <code>pom.xml</code> e JAR
Lista, fila, pilha, ABB e recursão	Código, testes e visualização nas telas
Inserir, remover e consultar	Operações expostas nas duas aplicações
Todas as operações Java presentes no C/C++ adaptado	Matriz e comparação de 56 operações HTTP
Tradução comentada ligada ao código real	Seções 3 a 8 deste documento
Exclusão das regras da Unidade 2	Sem FEFO, compatibilidade, hash de domínio ou roteirização
Comparação com HemoTrack original	Seção 1, nomes de classes e diferenças explícitas
Identificação e vínculo com código de sala	A conferir pelo aluno antes da submissão

Fontes e materiais de referência

HemoTrack(1).zip: modelos, controladores, serviços, repositórios, configuração e cinco templates HTML fornecidos pelo aluno; especialmente `ServicoBolsa`, `ServicoAlocacao` e `ServicoCompatibilidade`. Requisitos e imagens do enunciado da Unidade 1 fornecidos na conversa. Aulas 01 a 05 anteriormente anexadas: registros, ponteiros, estruturas lineares, recursão, árvores binárias e ABB.

Documentação oficial de requisitos do Spring Boot 4.1.1:

<https://docs.spring.io/spring-boot/system-requirements.html>. Código e licenças dos adaptadores C++:

<https://github.com/yhirose/cpp-httpplib/tree/v0.18.3> e <https://github.com/nlohmann/json/tree/v3.11.3>.